parse_fastq.py 사용법 및 설명서

- 위치: `lab11/src/parse_fastq.py`
- 목적: FASTQ 파일을 읽어서 read 개수와 첫 번째 read의 정보를 출력한다.
- 필요한 것: Python 3 (표준 라이브러리 `sys`만 사용, 추가 설치 없음)

---

1. 빠른 사용법

`lab11` 폴더에서 다음과 같이 실행한다.

```
python3 src/parse_fastq.py data/SRR1553607_1.fastq
```

형식:

```
python3 src/parse_fastq.py <FASTQ 파일 경로>
```

실행 결과 예시

```
read 개수: 203445
첫 read 이름: SRR1553607.1 1 length=101
첫 read 서열: GTTAGCGTTGTTGATCGCGACGCAACAACTGGTAAAGAATCTGGAAGAAGGATATCAGTTCAAACGCTCAAGCGAGATGATGGATATTTTTGAAC
GACTCA
첫 read 길이: 101
```

출력 항목	의미
read 개수	파일에 들어 있는 레코드(read) 수
첫 read 이름	첫 레코드 헤더에서 `@`를 뺀 부분
첫 read 서열	첫 레코드의 염기서열
첫 read 길이	첫 레코드 서열의 염기 수

`data/SRR1553607_1.fastq`는 813,780줄이므로 813,780 ÷ 4 = **203,445개** read가 나온다.

---

2. FASTQ 형식 복습

FASTQ 파일은 **4줄이 read 1개**이다.

```
@SRR1553607.1 1 length=101     <- 1줄: '@' + read 이름
GTTAGCGTTGTTGATCGCGACG...      <- 2줄: 염기서열
+SRR1553607.1 1 length=101     <- 3줄: '+' (구분자, 이름이 반복될 수도 있음)
```

```
BB@FFFFFHHHHHJJJJJJJJJ...      <- 4줄: 품질 점수 (서열과 길이가 같음)
```

---

3. 코드 설명

```
import sys

def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()

    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))

    return records

reads = read_fastq(sys.argv[1])
print("read 개수:", len(reads))
print("첫 read 이름:", reads[0][0])
print("첫 read 서열:", reads[0][1])
print("첫 read 길이:", len(reads[0][1]))
```

3-1. `import sys`와 `sys.argv[1]`

- `sys.argv`는 명령줄에 입력한 값들의 리스트이다.
- `python3 src/parse_fastq.py data/a.fastq` 로 실행하면
  - `sys.argv[0]` → `"src/parse_fastq.py"` (스크립트 이름)
  - `sys.argv[1]` → `"data/a.fastq"` (읽을 파일)

3-2. `read_fastq(filename)` 함수

코드	설명
`records = []`	결과를 담을 빈 리스트
`with open(filename) as f:`	파일을 열고, 블록이 끝나면 자동으로 닫는다
`lines = f.readlines()`	파일의 모든 줄을 리스트로 읽는다 (각 줄 끝에 `\n` 포함)
`for i in range(0, len(lines), 4):`	i = 0, 4, 8, … 처럼 **4씩 건너뛰며** 각 read의 첫 줄 위치를 가리킨다
`lines[i].rstrip()[1:]`	1줄: 끝의 줄바꿈 제거 후 맨 앞 `@` 한 글자를 잘라낸다 → 이름
`lines[i + 1].rstrip()`	2줄: 염기서열
`lines[i + 3].rstrip()`	4줄: 품질 점수 (3줄 `+`는 사용하지 않음)

`records.append((name, seq, qual))`	세 값을 튜플로 묶어 리스트에 추가
`return records`	모든 read의 리스트를 돌려준다

3-3. 반환값의 구조

```
reads = [
    ("SRR1553607.1 1 length=101", "GTTAGC...", "BB@FFF..."),   # reads[0]
    ("SRR1553607.2 2 length=101", "GGTGTA...", "?@;BDD..."),   # reads[1]
    ...
]
```

접근 방법	값
`reads[0]`	첫 번째 read (튜플)
`reads[0][0]`	첫 번째 read의 이름
`reads[0][1]`	첫 번째 read의 서열
`reads[0][2]`	첫 번째 read의 품질 점수
`len(reads)`	read 개수

---

4. 주의할 점과 한계

1. **파일 이름을 안 주면 오류**: `python3 src/parse_fastq.py` 처럼 실행하면 `sys.argv[1]`이 없어서 `IndexError`가 난다.
2. **줄 수가 4의 배수가 아니면 오류**: 파일 끝에 빈 줄이 있거나 잘린 파일이면 `lines[i + 3]`이 없어 `IndexError`가 날 수 있다.
3. **빈 파일이면 오류**: read가 0개이면 `reads[0]`에서 `IndexError`가 난다.
4. **메모리 사용**: `readlines()`는 파일 전체를 한 번에 메모리에 올린다. 수업 데이터(파일당 약 81만 줄)는 괜찮지만, 수 GB 크기의 FASTQ에서는 느리거나 메모리가 부족할 수 있다.
5. **형식 검사 없음**: 1줄이 `@`로, 3줄이 `+`로 시작하는지 확인하지 않는다. 형식이 잘못된 파일도 그대로 읽어 버린다.
6. **품질 점수는 읽기만 함**: `qual`은 저장되지만 화면에 출력하지는 않는다.

---

5. 수업 데이터

파일	줄 수	read 수
`data/SRR1553607_1.fastq`	813,780	203,445
`data/SRR1553607_2.fastq`	813,780	203,445

`_1`과 `_2`는 paired-end 시퀀싱의 정방향/역방향 read 파일이다.

```
python3 src/parse_fastq.py data/SRR1553607_2.fastq
```